

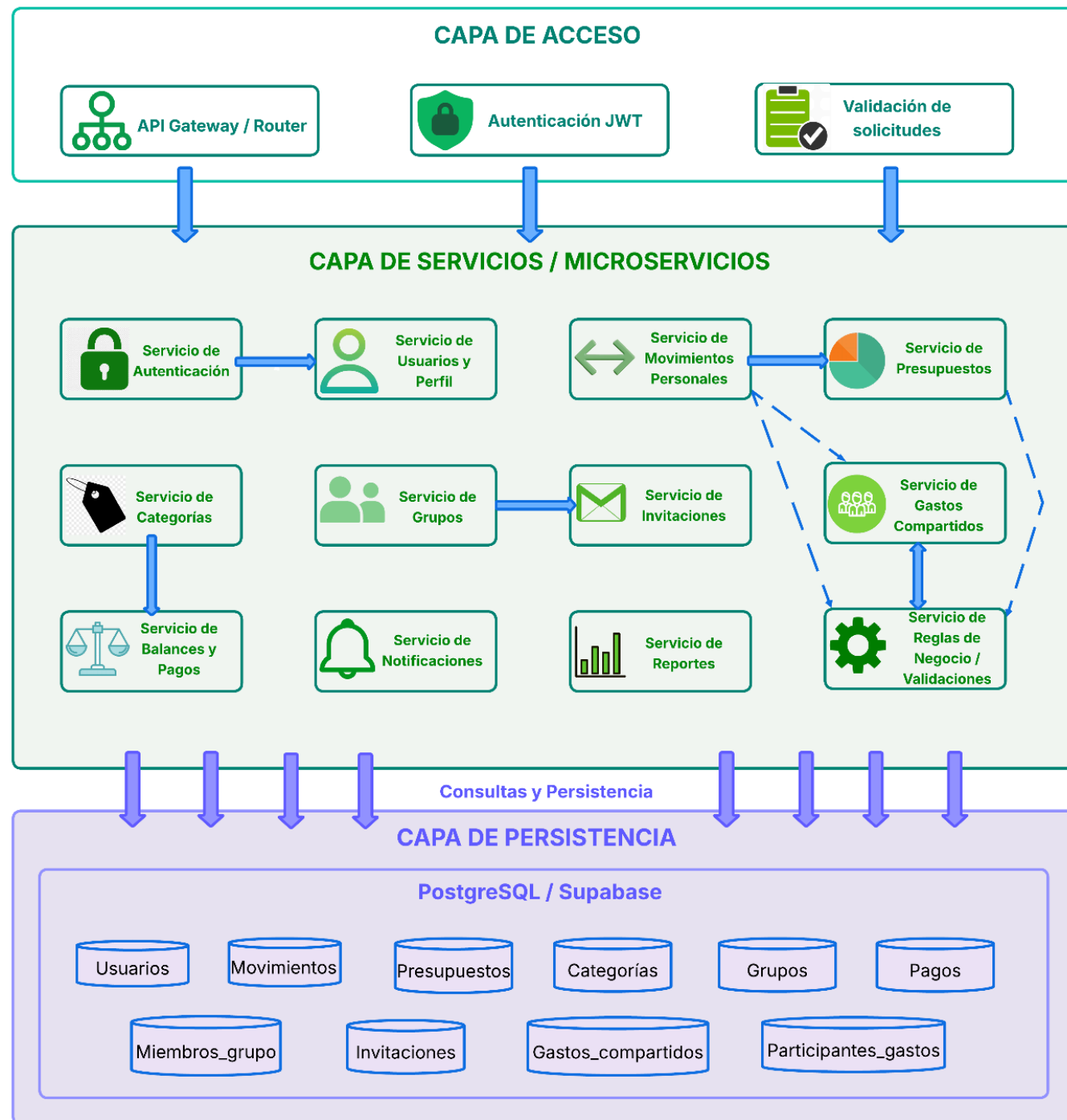
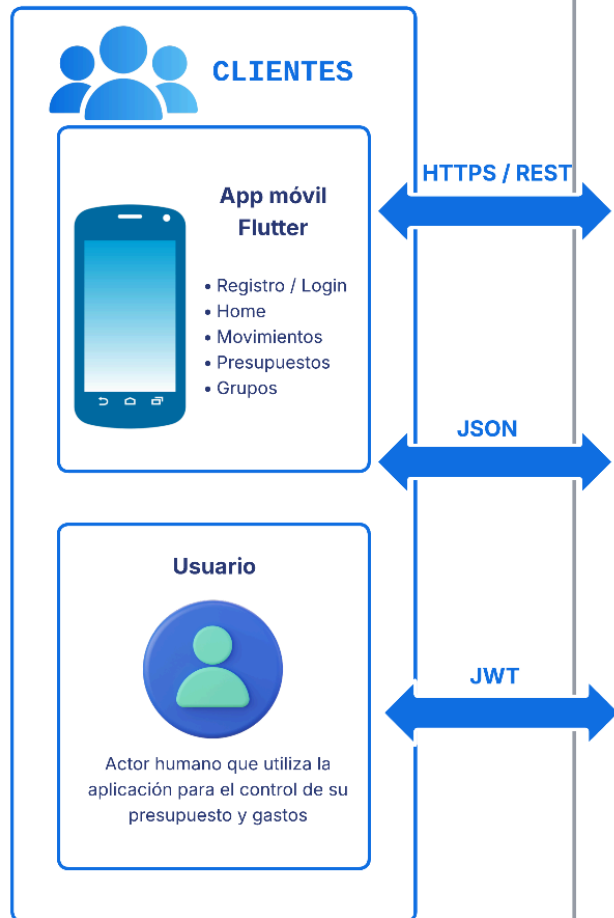
ACP

**Diseño de la arquitectura del sistema
(cliente-servidor)**

Diagrama de Arquitectura

Versión 1.0

Lima, 11 de Mayo del 2026



Descripción General

Se representa la arquitectura del sistema de la Aplicación de Control de Presupuestos, que se encuentra organizada bajo el modelo cliente-servidor. La arquitectura nos permite separar la aplicación móvil, la lógica del backend y a su vez la base de datos, logrando así que cada parte tenga una responsabilidad clara.

En este proyecto, el usuario interactúa desde la aplicación móvil que se encuentra desarrollada en Flutter. La aplicación envía solicitudes al backend mediante comunicaciones HTTPS/REST, usando los datos en formato JSON y la autenticación mediante JWT. El backend procesa las solicitudes, además aplica las reglas de negocio y consulta o actualiza la base de datos.

Objetivo de la arquitectura

Se busca:

- Separar la interfaz móvil de la lógica del negocio
- Evitar que el cliente acceda directamente a la base de datos
- Centralizar las validaciones en el backend
- Proteger las solicitudes mediante autenticación JWT
- Mantener separados los módulos personales y grupales
- Facilitar el mantenimiento y control de cambios
- Permitir que cada funcionalidad del sistema pueda ser revisada o modificada sin afectar innecesariamente a los demás módulos

Capas de la Arquitectura

Capa de clientes

Representa la parte visible del sistema, lo que interactúa directamente el usuario final

Componentes de esta capa

Usuario:

- Persona que utiliza la aplicación
- Puede registrar ingresos, gastos, presupuestos y grupos

- Acceso al sistema mediante registro o inicio de sesión
- Solo puede visualizar y administrar la información asociada a su cuenta

App móvil (Flutter):

- Contiene las pantallas principales de la aplicación: Registro / login - Home - Movimientos - Presupuestos - Grupos
- Tiene como función mostrar la interfaz, capturar los datos ingresados y enviar solicitudes al backend

Comunicación entre cliente y servidor

Entre la aplicación y el backend se observa la comunicación mediante:

- HTTPS
- REST
- JSON
- JWT

Cumplen las funciones:

- HTTPS permite que la comunicación viaje de forma segura
- REST define la forma en que la aplicación consume los servicios del backend
- JSON se utiliza como formato de intercambio de datos
- JWT permite identificar al usuario autenticado en cada solicitud

Capa de acceso

La capa de acceso se ubica en la parte superior del backend. Su función es recibir, organizar y validar las solicitudes que llegan desde la aplicación antes de enviarlas a los servicios internos

Componentes de la capa de acceso

API Gateway / Router:

- Recibe las solicitudes provenientes de la aplicación
- Dirige cada petición al módulo o servicio correspondiente
- Permite mantener ordenadas las rutas del backend

Autenticación JWT:

- Verifica si el usuario tiene una sesión válida
- Protege las funcionalidades que requieren autenticación
- Permite que el sistema identifique al usuario al realizar cada operación

Validación de solicitudes:

- Revisa que los datos enviados desde la app tengan una estructura correcta
- Evita que solicitudes incompletas o inválidas lleguen a la lógica principal
- Ayuda a reducir errores dentro del sistema

Esta capa actúa como primer filtro del backend. Antes de ejecutar una operación como registrar un movimiento, crear un presupuesto o ingresar a un grupo, el sistema comprueba que la solicitud sea válida y que el usuario tenga autorización.

Capa de servicios / microservicios

Es la lógica principal del sistema. Los servicios se muestran como módulos funcionales separados, esto no significa necesariamente que cada uno esté desplegado como un microservicio independiente, sino que la arquitectura los organiza como responsabilidades separadas dentro del backend

Servicios principales

De Autenticación:

- Gestiona el registro e inicio de sesión del usuario
- Valida credenciales
- Trabaja junto con el token JWT para permitir el acceso seguro

De Usuarios y Perfil:

- Administra la información básica del usuario
- Permite asociar cada operación al usuario autenticado
- Para controlar que cada usuario acceda solo a sus propios datos

De Movimientos Personales:

- Gestiona los ingresos y gastos
- Permite registrar, consultar, editar o eliminar movimientos

- Se relaciona con el presupuesto cuando un gasto puede afectar un límite definido por el usuario

De Presupuestos:

- Administra los presupuestos
- Válida montos, categorías y fechas de vigencia
- Permite calcular el avance del presupuesto con base en los gastos registrados

De Categorías:

- Administra para clasificar movimientos y presupuestos
- Permite organizar la información financiera del usuario

De Grupos:

- Gestiona la creación y consulta de grupos de gastos compartidos
- Administra la relación entre usuarios y grupos

De Invitaciones:

- Gestiona las invitaciones enviadas a otros usuarios
- Permite que un usuario invitado pueda aceptar o rechazar su incorporación a un grupo

De Gastos Compartidos:

- Permite registrar gastos dentro de un grupo
- Define quién pagó, quiénes participan y cómo se divide el monto
- Puede trabajar con división igualitaria, porcentual o personalizada

De Balances y Pagos:

- Calcula cuánto debe o cuánto le deben a cada integrante del grupo
- Registra pagos entre miembros para saldar deudas
- Clave para el módulo grupal del sistema

De Notificaciones:

- Sección de notificaciones de la aplicación

De Reportes:

- Funcionalidad proyectada para mostrar reportes financieros

De Reglas de Negocio / Validaciones:

- Centraliza validaciones importantes del sistema
- Revisa montos, fechas, categorías, usuarios, participantes y reglas de división
- Ayuda a mantener consistencia en las operaciones

Relaciones entre servicios

Estas conexiones representan dependencias lógicas entre módulos

Relaciones principales

Autenticación → Usuarios y Perfil

- Cuando un usuario se registra o inicia sesión, el sistema necesita vincular esa autenticación con su información de perfil

Movimientos Personales → Presupuestos

- Cuando se registra un gasto personal, el sistema puede verificar si existe un presupuesto activo para esa categoría

Grupos → Invitaciones

- Cuando un usuario crea un grupo puede invitar a otros miembros
- El servicio de grupos se conecta con el servicio de invitaciones

Gastos Compartidos → Reglas de Negocio / Validaciones

- Antes de guardar un gasto compartido el sistema debe validar el monto, los participantes y el tipo de división

Presupuestos → Reglas de Negocio / Validaciones

- Antes de crear o actualizar un presupuesto el sistema debe validar fechas, montos y categorías

Movimientos Personales → Reglas de Negocio / Validaciones

- Los movimientos deben pasar por validaciones como monto mayor a cero, categoría válida y datos obligatorios completos

Separación entre módulo personal y módulo grupal

Los ingresos, gastos y presupuestos personales no deben mezclarse con los gastos compartidos de grupos

Módulo personal:

- Movimientos personales, presupuestos, categorías, home y resumen financiero

Módulo grupal:

- Grupos, invitaciones, gastos compartidos, participaciones de gasto, balances y pagos

Esta separación evita errores en los cálculos, como por ejemplo, un gasto compartido no debe registrarse automáticamente como gasto personal, y un presupuesto personal no debe tomar en cuenta gastos grupales

Capa de persistencia

Almacena la información necesaria para el funcionamiento del sistema

Entidades principales

Usuarios:

- Almacena los datos de los usuarios registrados

Movimientos:

- Guarda los ingresos y gastos personales

Presupuestos:

- Registra los presupuestos definidos por el usuario

Categorías:

- Contiene categorías utilizadas para clasificar movimientos y presupuestos

Grupos:

- Guarda los grupos creados por los usuarios

Pagos:

- Registra pagos entre miembros de un grupo

Miembros_grupo:

- Relaciona usuarios con grupos

Invitaciones:

- Guarda invitaciones enviadas a usuarios para incorporarse a los grupos

Gastos_compartidos:

- Almacena los gastos registrados dentro de grupos

Participantes_gasto:

- Registra qué usuarios participaron en un gasto compartido y cuánto les corresponde pagar

Las operaciones de lectura y escritura pasan por el backend, el cual valida primero la solicitud y luego realiza la consulta correspondiente. Lo cual ayuda a mantener la seguridad y el control de acceso

Flujo general de funcionamiento

1. El usuario interactúa con la aplicación
2. La app captura la información ingresada
3. La app envía una solicitud al backend usando HTTPS / REST
4. La solicitud viaja con datos en formato JSON
5. Si la operación requiere autenticación, se envía el token JWT
6. La capa de acceso recibe la solicitud
7. El sistema valida el token
8. La solicitud se dirige al servicio correspondiente
9. El servicio aplica la lógica de negocio
10. Si se requiere guardar o consultar información, el servicio accede a PostgreSQL / Supabase
11. La base de datos devuelve la información solicitada
12. El backend responde a la aplicación móvil
13. La app muestra el resultado al usuario

Ejemplo de un gasto personal

1. El usuario ingresa a la app
2. Selecciona la opción para registrar un nuevo movimiento
3. Completa el formulario con monto, categoría, fecha, descripción y método de pago
4. La app envía la solicitud al backend
5. El backend valida el JWT del usuario
6. El servicio de movimientos personales revisa los datos enviados
7. El servicio de reglas de negocio valida que el monto sea correcto y que la categoría exista
8. Si el movimiento es un gasto, el servicio de presupuestos puede verificar si existe un presupuesto activo
9. El sistema guarda el movimiento en la base de datos
10. La app muestra la confirmación al usuario

Justificación de la arquitectura empleada

- Permite separar responsabilidades entre cliente, backend y base de datos
- Facilita el mantenimiento del sistema
- Permite organizar el backend por módulos funcionales
- Ayuda a mantener separados los datos personales y grupales
- Permite controlar mejor los cambios durante el desarrollo
- Facilita la documentación técnica del proyecto
- Es coherente con una aplicación que requiere autenticación, validaciones y persistencia de datos

Conclusión

La división por servicios permite visualizar mejor las responsabilidades del backend aunque estos servicios pueden entenderse como módulos lógicos dentro de FastAPI y no necesariamente como microservicios desplegados por separado. Esta representación muestra los módulos principales del sistema, respeta la separación entre información personal y grupal, y además facilita el control de configuración del software durante el desarrollo.